

《程序设计实习》项目报告

2D 大逃杀游戏：SteveRun

小组成员：吉佑安 樊轩楷

GitHub 仓库: [cxsjsx_very_QuTe](#)

2026 年 7 月 3 日

目录

1 程序功能介绍

本项目是一款名为 **SteveRun** 的 2D 大逃杀类动作游戏。游戏融入了经典沙盒游戏《Minecraft》的诸多元素与彩蛋，玩家需要在源源不断的敌人包围中生存并尽可能获取高分。程序功能主要分为主菜单界面设计与游戏本体机制两大部分。

1.1 UI 设计与主菜单功能

主界面的设计清晰且富有交互性，背景配备了主菜单 BGM。界面中包含了 Steve 头上的分数记录以及游戏标题右下方的 Tips 小彩蛋（致敬 Minecraft）。左下角提供了四个核心选项功能：

- **开始游戏**：点击后弹出游戏主窗口，直接进入大逃杀对战。
- **设置**：包含游戏难度选择以及全局音量调节功能。
- **皮肤选择**：打开装备库页面，提供游戏主题选择以及部分装备的皮肤自定义选项。配置完毕后按 **Esc** 或点击关闭按钮即可，系统会自动保存用户的搭配。
- **模式选择**：提供“计时模式”与“无限模式”。计时模式下，玩家可选择 30 到 300 秒的生存时间上限；无限模式则不限时长，直到玩家碰到敌人死亡为止。

此外，我们通过本地 `.bin` 文件的读写，实现了偏好记忆功能。用户的皮肤外观、音量大小、难度、模式和时长等配置均会持久化保存，下次打开程序时可无缝恢复。

1.2 游戏本体机制

游戏本体构建了一个富有张力的生存战场，涵盖了背景画面与动态音效。

- **玩家控制与交互：** 玩家通过鼠标操控角色（Steve）的移动方向，并决定技能释放的瞄准方向。游戏地图边界设有“边框传送门”，玩家和敌人均可通过边缘实现屏幕穿梭（Screen Wrap）。
- **技能系统：** 技能球在地图上随机生成，玩家拾取后可使用。共包含五种特色技能：近战长矛、连弩、追踪烟花弩、不死图腾护盾、*TNT* 剑气，为战斗提供了丰富的策略选择。
- **敌人与生成机制：** 敌人会在玩家周围随机生成，并伴有生成预警提示。敌人具备追踪 AI，会自动计算并选择最短路径（包括穿过边框传送门）向玩家移动。为了增加挑战性，我们设计了**阵型生成系统**，敌人可以按照“环形包围圈”、“弹性方块”、“延迟发射箭头”等复杂阵型生成，极大地提升了游戏的可玩性和难度。
- **UI 呈现与结算：** 游戏界面左上角设有计分计时板，实时记录杀敌数量及已过/剩余时长。对局结束后弹出结算界面（含死亡界面和计时结束界面），展示所得最终分数；点击“EXIT”退回主菜单后，若打破记录，将更新最高分显示。

2 项目各模块与类设计细节

本程序基于 C++ 与 Qt 框架开发，遵循了高内聚、低耦合的面向对象设计原则。项目代码在逻辑上划分为三大核心模块：

2.1 窗口与视图模块 (Window Module)

该模块主要负责程序的界面渲染与场景调度，包含以下核心类：

- **MainWindow：** 主窗口类，作为程序的入口。负责承载主菜单 UI，处理用户在“开始、设置、模式”等选项间的导航，并负责启动游戏视图。
- **GameView：** 游戏核心视图类。作为游戏引擎的承载者，管理游戏的主循环（Game Loop）、场景渲染、用户输入（鼠标追踪机制），并统筹对象间的碰撞检测与事件分发。
- **skinselect：** 独立的设置/皮肤选择窗口类。负责展示装备库，处理界面上的按钮与皮肤切换逻辑。

2.2 游戏对象模块 (Game Object Module)

所有参与游戏内交互的实体都属于此模块，主要继承自 Qt 的图形项基类，并实现了各自的物理和渲染逻辑：

- **玩家类 (Player):** 封装了玩家的坐标、生命值、状态机以及移动算法，包含响应外部输入触发技能释放的接口。
- **敌人类 (Enemy, Formation):** Enemy 类封装了敌人的自动寻路逻辑与属性; Formation 类则作为生成控制器，通过工厂模式思想，在特定时机批量构造并布局处于阵型中的敌人实体。
- **实体技能类 (Skill Entities):** 将玩家的五大技能与敌人的各种攻击手段(如 Arrow、DeathVFX 等) 抽象为独立的实体类，封装了各自的飞行轨迹、生命周期及伤害判定逻辑。

2.3 数据通信与“胶水”模块 (Glue Module)

为了避免多窗口之间复杂的指针传递，我们设计了静态数据枢纽：

- **DataCarrier:** 该类作为全局的数据承载器与状态机。集中存储玩家偏好、分数记录、当前难度及游戏模式等静态数据。同时，它负责实现底层的文件 I/O 流，完成与 .bin 文件的序列化和反序列化操作，完美充当了 UI 模块与游戏业务逻辑之间的桥梁。

3 小组成员分工情况

本项目由两位组员通力协作完成，任务分配兼顾了框架搭建与功能实现：

- **吉佑安**
 - **框架与 UI 开发:** 负责窗口模块的半数工作，与樊轩楷平分 MainWindow 与 GameView 的核心逻辑编写；独立完成了 skinselect 界面类的设计与开发。
 - **数据模块:** 设计并实现了“胶水”模块 DataCarrier，打通了游戏数据在全局的流转，并实现了偏好设置的本地文件记忆功能。
 - **实体逻辑协助:** 参与了部分游戏实体的逻辑梳理和代码编写，共同完善游戏对象模块。
- **樊轩楷**
 - **项目奠基与核心系统:** 负责项目早期架构的奠基，构建了 Qt 图形场景的渲染基础。与吉佑安平分并协同开发了 MainWindow 与 GameView。
 - **技能与战斗设计:** 承担了重度游戏逻辑，编写了部分玩家技能逻辑以及全部敌人的技能与阵型行为。
 - **调优与重构:** 在开发过程中不定时对战斗系统、参数平衡和实体逻辑进行反复修改与迭代优化。

4 项目总结与反思

在本次《程序设计实习》中，我们将 C++ 面向对象的高级特性与 Qt 框架（如 QGraphicsView / QGraphicsScene 体系）深度结合，成功从零打造了一款具备完整生命周期的 2D 大逃杀游戏。游戏不仅具备完善的 UI 逻辑（如数据序列化存储），在游戏性上也通过“阵型生成”、“边界传送”等机制实现了微创新，最终在本地顺利运行并展现了较好的性能。

在开发过程中，我们遇到了不少挑战，也获得了宝贵的经验：

1. **内存管理与性能优化：**作为一款大逃杀游戏，同屏存在大量的弹幕、技能特效与敌人实体。初始阶段面临着由于频繁 `new` 和无序 `delete` 导致的内存泄漏与卡顿。通过深入理解 Qt 的对象树机制并结合 C++ 智能指针/合理的对象池思路，我们确保了所有继承自 `QGraphicsItem` 的实体在生命周期结束（如飞出屏幕边界、击中目标）后能被安全析构，保障了帧率稳定。
2. **复杂碰撞检测与事件分发：**在使用鼠标实时计算角度、控制人物技能时，坐标系的转换（Scene、View、Item 坐标系）曾带来困难。我们查阅文档，熟练掌握了各种坐标映射方法。同时将碰撞检测集中在 `GameView` 或场景的特定 `timer` 中统一调度，使得判定更为精准。
3. **Git 与团队协作研发：**在开发 `MainWindow` 和 `GameView` 这两个核心模块时，我们采取了“平分工作”的方式。由于代码交集大，如何利用 GitHub 避免代码冲突是一大考验。在不断摸索中，我们学会了合理分摊函数接口、使用 Git 指令、规范化 Commit 提交。这极大地提升了我们的软件工程素养。

综上所述，本项目不仅锻炼了我们运用 C++ 解决复杂工程问题的能力，更让我们体会到了游戏开发从“机制设计”到“代码落地”再到“打磨优化”的完整快乐。对于未来的改进，我们希望能进一步抽象实体框架、增加更丰富的场景地图与局域网联机功能。